# DBMS PRACTICAL SOLUTIONS

## Set 1: College Management System

### 1. Create Database & Tables

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    DeptID INT
);

CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50)
);
```

### 2. Insert & Update

```
INSERT INTO Student VALUES
(1,'Amit',101),
(2,'Riya',102),
(3,'John',101),
(4,'Sara',103),
(5,'Neha',102);

INSERT INTO Course VALUES
(1,'DBMS'),
(2,'OS'),
(3,'Java'),
(4,'Python'),
(5,'AI');

UPDATE Student
```

```
SET DeptID = 104
WHERE StudentID = 1;
```

### 3. GROUP BY & HAVING

```
SELECT DeptID, COUNT(*) AS TotalStudents
FROM Student
GROUP BY DeptID
HAVING COUNT(*) > 1;
```

### 4. INNER JOIN

(*Assuming StudentID = CourseID for demonstration*)

```
SELECT S.Name, C.CourseName
FROM Student S
INNER JOIN Course C
ON S.StudentID = C.CourseID;
```

---

## Set 2: Employee Management System

### 1. Create Table

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2),
    DeptID INT
);
```

### 2. Insert & Update

```
INSERT INTO Employee VALUES
(1,'A',30000,10),
(2,'B',40000,20),
(3,'C',35000,10),
(4,'D',50000,30),
(5,'E',45000,20);

UPDATE Employee
```

```
SET Salary = Salary + 5000
WHERE DeptID = 10;
```

**3. Average Salary**

```
SELECT DeptID, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY DeptID;
```

**4. Subquery**

```
SELECT Name, Salary
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

---

**Set 3: Department & Employee**

**1. Create Tables**

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

**2. Insert Records**

```
INSERT INTO Department VALUES
(1,'HR'),(2,'IT'),(3,'Sales');

INSERT INTO Employee VALUES
(1,'A',1),
(2,'B',2),
```

```
(3,'C',1),
(4,'D',3);
```

**3. Count Employees**

```
SELECT DeptID, COUNT(*) AS TotalEmployees
FROM Employee
GROUP BY DeptID;
```

**4. LEFT JOIN**

```
SELECT D.DeptName, E.Name
FROM Department D
LEFT JOIN Employee E
ON D.DeptID = E.DeptID;
```

---

**Set 4: Online Shopping**

**1. Create Table**

```
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    Name VARCHAR(50),
    Price DECIMAL(10,2),
    Category VARCHAR(50)
);
```

**2. Insert & Delete**

```
INSERT INTO Product VALUES
(1,'Laptop',70000,'Electronics'),
(2,'Phone',30000,'Electronics'),
(3,'Shoes',2000,'Fashion'),
(4,'Watch',5000,'Fashion'),
(5,'TV',40000,'Electronics');

DELETE FROM Product
WHERE Price < 3000;
```

**3. GROUP BY**

```
SELECT Category, SUM(Price) AS TotalPrice
FROM Product
GROUP BY Category;
```

### 4. Most Expensive Product

```
SELECT *
FROM Product P
WHERE Price = (
    SELECT MAX(Price)
    FROM Product
    WHERE Category = P.Category
);
```

---

## Set 5: Banking System

### 1. Create Tables

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    Name VARCHAR(50)
);

CREATE TABLE Account (
    AccountID INT PRIMARY KEY,
    CustomerID INT,
    Balance DECIMAL(10,2),
    FOREIGN KEY (CustomerID) REFERENCES
Customer(CustomerID)
);
```

### 2. Insert & Update

```
INSERT INTO Customer VALUES (1,'A'),(2,'B');

INSERT INTO Account VALUES
(101,1,5000),
(102,1,7000),
(103,2,8000);
```

```
UPDATE Account
SET Balance = Balance + 1000
WHERE CustomerID = 1;
```

### 3. Total Balance

```
SELECT CustomerID, SUM(Balance) AS TotalBalance
FROM Account
GROUP BY CustomerID;
```

### 4. RIGHT JOIN

```
SELECT C.Name, A.AccountID, A.Balance
FROM Customer C
RIGHT JOIN Account A
ON C.CustomerID = A.CustomerID;
```

---

## Set 6: Library System

### 1. Create Tables

```
CREATE TABLE Book (
    BookID INT PRIMARY KEY,
    Title VARCHAR(50)
);

CREATE TABLE Issue (
    IssueID INT PRIMARY KEY,
    BookID INT,
    MemberID INT,
    FOREIGN KEY (BookID) REFERENCES Book(BookID)
);
```

### 2. Insert

```
INSERT INTO Book VALUES (1,'DBMS'),(2,'OS');

INSERT INTO Issue VALUES
(1,1,101),
```

```
(2,1,102),
(3,2,103);
```

### 3. Nested Query

```
SELECT BookID
FROM Issue
GROUP BY BookID
HAVING COUNT(*) > 1;
```

### 4. INTERSECT

```
SELECT BookID FROM Book
INTERSECT
SELECT BookID FROM Issue;
```

---

## Set 7: Hospital System

### 1. Create & ALTER

```
CREATE TABLE Patient (
    PatientID INT PRIMARY KEY,
    Name VARCHAR(50),
    DoctorName VARCHAR(50)
);

ALTER TABLE Patient ADD Age INT;
```

### 2. Insert & Delete

```
INSERT INTO Patient VALUES
(1,'A','Dr.X',25),
(2,'B','Dr.Y',30),
(3,'C','Dr.X',40);

DELETE FROM Patient
WHERE Age > 35;
```

### 3. Correlated Subquery

```
SELECT *
FROM Patient P1
WHERE EXISTS (
    SELECT 1 FROM Patient P2
    WHERE P1.DoctorName = P2.DoctorName
    AND P1.PatientID <> P2.PatientID
);
```

**4. MINUS**

```
SELECT Name FROM Patient
MINUS
SELECT Name FROM Patient WHERE DoctorName = 'Dr.X';
```

---

**Set 8: University Database**

**1. Create Tables**

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

**2. Insert & Update**

```
INSERT INTO Department VALUES (1,'CS'),(2,'IT');

INSERT INTO Student VALUES
(1,'A',1),
(2,'B',2);

UPDATE Student SET Name='C' WHERE StudentID=1;
```

### 3. ORDER BY + HAVING

```
SELECT DeptID, COUNT(*) AS Total
FROM Student
GROUP BY DeptID
HAVING COUNT(*) > 0
ORDER BY Total DESC;
```

### 4. FULL JOIN

```
SELECT *
FROM Student
FULL JOIN Department
ON Student.DeptID = Department.DeptID;
```

---

## Set 9: School Result

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT
);

INSERT INTO Student VALUES
(1,'A',80),(2,'B',60),(3,'C',90);

UPDATE Student SET Marks=70 WHERE StudentID=2;

DELETE FROM Student WHERE Marks < 65;

SELECT SUM(Marks), AVG(Marks) FROM Student;

SELECT *
FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

---

## Set 10: Inventory System

```sql
CREATE TABLE Inventory (
    ProductID INT PRIMARY KEY,
    Name VARCHAR(50),
    Quantity INT
);

CREATE TABLE Supplier (
    SupplierID INT PRIMARY KEY,
    ProductID INT
);

ALTER TABLE Inventory ADD Price INT;

SELECT ProductID, SUM(Quantity)
FROM Inventory
GROUP BY ProductID;

SELECT *
FROM Inventory I
JOIN Supplier S
ON I.ProductID = S.ProductID;
```

---

**Set 11: Airline System**

```sql
CREATE TABLE Flight (
    FlightID INT PRIMARY KEY,
    FlightName VARCHAR(50),
    Source VARCHAR(50),
    Destination VARCHAR(50)
);

CREATE TABLE Passenger (
    PassengerID INT PRIMARY KEY,
    Name VARCHAR(50),
    FlightID INT,
    FOREIGN KEY (FlightID) REFERENCES Flight(FlightID)
);

SELECT FlightID, COUNT(*)
FROM Passenger
```

```
GROUP BY FlightID
HAVING COUNT(*) > 1;

SELECT *
FROM Passenger P
INNER JOIN Flight F
ON P.FlightID = F.FlightID;
```

---

**Set 12: Hotel System**

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    Name VARCHAR(50)
);

CREATE TABLE Booking (
    BookingID INT PRIMARY KEY,
    CustomerID INT,
    RoomType VARCHAR(50),
    Amount DECIMAL(10,2)
);

SELECT CustomerID, SUM(Amount)
FROM Booking
GROUP BY CustomerID;

SELECT *
FROM Customer C
LEFT JOIN Booking B
ON C.CustomerID = B.CustomerID;
```

---

**Set 13: Examination System**

```
CREATE TABLE Result (
    StudentID INT,
    Name VARCHAR(50),
    Subject VARCHAR(50),
    Marks INT
);
```

```
DELETE FROM Result WHERE Marks < 40;

SELECT Subject, AVG(Marks)
FROM Result
GROUP BY Subject;

SELECT *
FROM Result
WHERE Marks > (SELECT AVG(Marks) FROM Result);
```

---

**Set 14: Transport System**

```
CREATE TABLE Vehicle (
    VehicleID INT PRIMARY KEY,
    Type VARCHAR(50)
);

CREATE TABLE Driver (
    DriverID INT PRIMARY KEY,
    Name VARCHAR(50),
    VehicleID INT,
    FOREIGN KEY (VehicleID) REFERENCES
Vehicle(VehicleID)
);

SELECT VehicleID, COUNT(*)
FROM Driver
GROUP BY VehicleID;

SELECT *
FROM Vehicle V
RIGHT JOIN Driver D
ON V.VehicleID = D.VehicleID;
```

---

**Set 15: Course Enrollment**

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
```

```
    Name VARCHAR(50)
);

CREATE TABLE Enrollment (
    StudentID INT,
    CourseID INT
);

ALTER TABLE Enrollment ADD EnrollDate DATE;

SELECT *
FROM Student S
WHERE (
    SELECT COUNT(*)
    FROM Enrollment E
    WHERE E.StudentID = S.StudentID
) > 1;

SELECT *
FROM Student S
JOIN Enrollment E
ON S.StudentID = E.StudentID;
```